

会计档案四性检测与元数据合规 —— 三份详细方案

编制：CodeBuddy · 面向仓库 xiaoluxiaolu/deepCode (会计档案管理系统 financeDA)

日期：2026-08-24

依据：DA/T 94-2022 《电子会计档案管理规范》、DA/T 70—2018、GB/T 18894-2016、《会计档案管理办法》（财政部/国家档案局 79号令）

范围：仅出具详细方案（不写代码），涵盖

① 《各凭证类型元数据 vs DA/T 94 逐项符合性差异表》；

② 断点①/③ 代码修复 PR 方案；

③ "原始凭证四性检测"接入方案（isRequired 落地到后端 + 检测项）。

第一部分 《各凭证类型元数据 vs DA/T 94 逐项符合性差异表》详细方案

1.1 问题背景与目标

系统内存在四层凭证/档案类型体系，四性检测与归档的实体对象各不相同，元数据合规性必须分层评估，不能混为一谈：

层级	类型体系	定义位置	数量
① 档案门类	KP会计凭证 / KB会计账簿 / FB财务报表 / QT其他	finance-model-current.xml typeCodeList	4 类
② 记账凭证子类	收款 / 付款 / 转账 / 通用	用友同步 voucherCategory	4 类
③ 账簿/报表子类	总账/明细账/日记账 + 月/季/年报	subType 、 reportCategory/Period	—
④ 原始凭证（附件级）	外来/自制/特殊，下设发票/银行/存货/费用/薪酬等	sourceDocument.ts SOURCE_DOC_TYPE_TREE	声明 96 种，实际叶节点 仅约 60 种

关键事实：真正承载四性检测与归档管理的是 ①②（finance:record 记账凭证件）；④ 原始凭证是依附记账凭证的附件级实体，目前不进四性检测引擎（本方案第三部分专门解决）。

元数据合规参照物 = DA/T 94-2022 附录A（件级 M1-M49、卷级 V1-V20、卷件关联 VA1-VA6）+ DA/T 39/42（盒级 B1-B29）。系统已用 metadataCatalog.ts 完整声明上述元数据目录。

1.2 差异表设计思路（"三层三对照"）

为做到"逐项、可复核、可落地"，差异表采用三层对照结构：

- 层1 实体类型层 —— 对照"系统有哪些类型"
- 层2 元数据顶层 —— 对照"每个元数据项是否符合 DA/T 94 的定义/必选/取值范围"
- 层3 落地映射层 —— 对照"声明的元数据是否在内容模型(finance-model-current.xml)/后端读写中真实落地"

每行结论统一为五档判定，避免模糊表述：

判定	含义	图例
□合规	定义、必选属性、值域、落地映射与标准一致	通过
□部分合规	有定义但落地/取值不完整	需整改
□不合规/缺失	无定义或定义与标准冲突	需补建
○ 不适用	标准不要求该实体具备的元数据	跳过
□自研扩展	系统自加、非标准强制（标注可选）	保留但注明

1.3 差异表主体（方案交付的完整表格结构）

1.3.1 件级（记账凭证 `finance:record`）对照 DA/T 94 附录A（M1-M49）

DA/T94 元数据	标准必选	系统映射字段	落地状态	判定	差异说明
M1 聚合层次	必选	archiveType (门类)	内容模型有、读取有	□	系统以"类别"粒度呈现
M2 来源(复合)	必选	—（仅以 sourceSystem 标记）	仅部分映射	□	来源复合对象未展开为结构化子项
M3 档案馆名称	必选	—	未落地	□	元数据目录声明了，内容模型无字段
M4 档案馆代码	必选	—	未落地	□	同上
M5 全宗名称	必选	fondsCode	有（fonds 维度）	□	以全宗号承载
M6 全宗号	必选	fondsCode	有	□	一致
M7 立档单位名称	必选	—（单位配置在 Unit 域）	部分	□	通过单位配置关联，未入件属性
M8 类别号	必选	archiveType	有	□	与门类一致
M9 室编案卷号	必选	volumeCode	有	□	卷级关联
M10 馆编案卷号	可选	—	未落地	○	可选，暂不适用
M11 室编件号	必选	volumeItemNo	有	□	
M12 馆编件号	可选	—	未落地	○	
M13 档号	必选	archiveCode	有（GD-1-03 校验）	□	格式复合，检测项覆盖
M14 题名	必选	cm:name / title	有	□	
M15 日期	必选	voucherDate	有	□	

DA/T94 元数据	标准 必选	系统映射字段	落地状态	判定	差异说明
M16 文件编号	可选	documentNo	有	□	
M17 责任者	必选	preparer	有	□	
M18 附件	可选	attachedBillCount	部分	□	附件明细未结构化
M19 密级	可选	securityLevel	有（GD-4-03 校验）	□	
M20 保管期限	必选	retention	有	□	
M21 摘要	可选	recordRemark / description	有	□	
M22 格式信息	必选	contentType	有	□	
M23 计算机文件名	必选	name	有	□	
M24 计算机文件大小	必选	sizeInBytes	有	□	
M25 计算机文件格式	必选	contentType	有	□	
M26 创建时间	必选	createdAt	有	□	
M27 哈希值	必选	digitalHash	字段有但默认禁用、仅判非空	□	GD-1-02 不重算哈希，仅判存在（见断点/第三部分）
M28 电子签名	条件 可选	signatureVerified	组件级有、件级未落库	□	未做可靠验签
M29 会计年度	必选	year	有（GD-3-03 校验）	□	
M30 会计资料形式	必选	archiveType	有	□	
M31 凭证号	条件 可选	voucherNo	有（GD-2-02/03 校验）	□	
M32 起始日期	可选	voucherDate	部分	□	
M33 终止日期	可选	—	未落地	○	
M34 币种	可选	—	未落地	○	
M35 金额合计	可选	amount	有（GD-2-05 校验）	□	
M36 页数	可选	pageCount	部分	□	
M37 存储位置	必选	volumeId / boxNo	有	□	
M38 脱机载体编号	条件 可选	carrierType	部分	□	未落具体编号
M39 关联档案号	可选	parentRecordId	有	□	
M40 机构人员类型	必选	preparer (角色)	部分	□	仅制单人角色
	必选		有	□	

DA/T94 元数据	标准 必选	系统映射字段	落地状态	判定	差异说明
M41 机构人员名称		preparer / auditor / tallyMan			
M42 机构人员代码	可选	—	未落地	○	
M43 业务类型	必选	sourceSystem	部分	□	
M44 业务名称	必选	—	未落地	□	业务活动未建模
M45 业务描述	可选	recordRemark	有	□	
M46 业务时间	必选	createdAt	有	□	
M47 关系类型	必选	parentRecordId (隐含)	部分	□	关系未显式类型化
M48 关系实体	必选	—	部分	□	
M49 关联实体	必选	—	部分	□	

件级结论：M1-M49 中约 30 项已真实落地（□）、约 15 项部分合规（□）、约 4 项缺失（□ M3/M4 档案馆代码、M44 业务名称）。整体具备合规骨架，但真实性与业务实体类元数据是短板。

1.3.2 卷级（finance:volume）对照 DA/T 94/39（V1-V20）

DA/T 元数据	标准必选	系统字段	判定	说明
V1 全宗号	必选	fondsCode	□	
V2 档案门类代码	必选	categoryCode =KU	□	
V3 年度	必选	year	□	
V4 类别号	必选	archiveType	□	
V5 案卷号	必选	volumeNo	□	
V6 案卷档号	必选	volumeCode	□	组合规则一致
V7 案卷题名	必选	title	□	
V8 起止日期	必选	dateRange	□	
V9 保管期限	必选	retention	□	
V10 卷内件数	必选	volumeTotalItems	□	GD 卷级 volume-count-match 校验
V11 卷内总页数	必选	totalPages	□	
V12 立档单位	必选	archivalUnit	□	
V13 立卷人	必选	filer	□	
V14 立卷日期	必选	filingDate	□	
V15 检查人	可选	inspector	□	
V16 检查日期	可选	inspectionDate	□	
V17 存放位置	条件可选	storageLocation	□	库房-架-层-位

DA/T 元数据	标准必选	系统字段	判定	说明
V18 数字化状态	条件可选	digitizationStatus	□	
V19 数字化副本哈希	条件可选	digitalHash	□	字段有、默认禁用
V20 备注	可选	remarks	□	

卷级结论：V1-V20 全覆盖且基本合规（□18 项 / □1 项），是四层里合规度最高的层级。

1.3.3 盒级（`finance:archiveBox`）对照 DA/T 39/42/94（B1-B29）

分类	系统字段	判定	说明
B1-B3 基础标识	BoxNo/FondsCode/CategoryCode	□	与档号体系一致
B4-B7 分类合规	年度/二级类别/保管期限/组织机构	□	对齐 DA/T 42 同一年度/类别/期限装盒
B8-B15 内容范围	起止案卷号/件号/卷数/件数/起止日期/月份/总册/本册	□	封面脊背必填项
B16-B19 物理位置	库房/架/层/位	□	自研扩展，非标准强制，标注可选
B20-B26 流程管理	装盒人/日期/整理/审核/状态/清册/备考	□□	流程留痕，部分为标准扩展
B27-B29 双套制	电子批次/介质标识/校验状态	□	自研扩展，条件可选

盒级结论：B1-B15 合规度良好；B16-B29 多为自研扩展（□），须在元数据目录中显式标注“自研扩展/非标准强制”，与标准项区分，避免被误判为标准强制项。

1.3.4 原始凭证（`finance:sourceDocument`，附件级）—— 本次最薄弱环节

DA/T 94 对原始凭证虽无独立附录，但依据 M18 附件元数据 + 会计档案法律要件（发票代码/号码/税号等），需逐类型评估。系统采用“9 公共字段 + extFields(JSON)”设计，存在 4 个关键缺陷：

#	缺陷	现状	影响	判定
a	类型数与声明不符	声明 96 种，实际叶节点仅约 60 种	配置面虚高	□
b	扩展字段非结构化	<code>finance:extFields</code> 单一 JSON 文本，未展开为内容模型属性	无法做属性级检索/校验	□
c	<code>isRequired</code> 只在前端生效	后端 <code>SourceDocService.create</code> 未写入 <code>extFields</code> 、无必填校验	发票代码/税号等法律要件缺失时检测不到	□
d	不进四性检测引擎	<code>InspectionService</code> 只针对 <code>finance:record</code>	最易篡改的附件无任何检测	□

逐类型差异抽样（发票类，最高合规要求）：

法律要件字段	前端 <code>extFieldDefs.isRequired</code>	内容模型	后端写入	四性检测	判定
发票代码 <code>invoiceCode</code>	□true	□无属性	□不写	□不检	□
发票号码 <code>invoiceNo</code>	□true	□	□	□	□

法律要件字段	前端 extFieldDefs.isRequired	内容模型	后端写入	四性检测	判定
校验码 checkCode	☐true	☐	☐	☐	☐
销售方税号 sellerTaxId	☐true	☐	☐	☐	☐
购买方税号 buyerTaxId	☐true	☐	☐	☐	☐
金额(不含税/税额/税率)	☐true	☐	☐	☐	☐

原始凭证结论：严格意义上不完全合规。前端声明的必填约束（isRequired）完全是“纸面约束”，后端既未持久化 extFields，也未做任何校验，更未接入四性检测。这是本方案第三部分要解决的。

1.4 差异表交付物（最终产出）

本部分最终交付一份可机读的 Excel/CSV 差异表，列结构建议：

实体层级 | 实体类型 | DA/T94元数据ID | 标准名称 | 标准必选 | 系统映射字段 | 内容模型落点 | 后端读写 | 四性检测覆盖 | 判定 | 差异

每条记录对应一个“实体类型 × 元数据项”组合，用于支撑：

- 元数据合规审计报告；
- 缺失字段的补建 backlog；
- 四性检测必填清单的动态扩充（见第三部分）。

优先级排序建议：

1. P0：原始凭证 extFields 持久化 + isRequired 落地（法律要件）；
2. P1：件级 M3/M4/M44 缺失字段补建；M27 哈希真校验；
3. P2：盒级 B16-B29 自研扩展标注；
4. P3：业务实体类元数据（M40-M49）建模完善。

第二部分 断点①/③ 代码修复 PR 详细方案

对应前次评审确认的两个必须修复（影响正确性）断点：

- 断点①：前端 dtoToRecord 硬编码 checks={real:false,...}，后端结果读不到 → 前端统计永远 0%；
- 断点③：配置页“立即执行检测”传 phase:'manual-config-page'，库里无此环节 → 空结果 → 假通过。

2.1 断点①：打通前端读取四性检测结果

2.1.1 现状与问题根因

- 后端 InspectionService.finishReport 已把结果写进：
- ams_inspection_report 表（ real/complete/usable/safe/all_pass ）；

- 节点 aspect `finance:checkReal/checkComplete/checkUsable/checkSafe/checkedAt/checkReportRef` (`finance-model-current.xml` 的 `finance:fourChecked`)。
- 前端 `recordService.ts:311` 的 `dtoToRecord` 硬编码：
`ts checks: { real: false, complete: false, usable: false, safe: false }, checkDetails: [],`
- 后端 `RecordDto` 未透出 `checkReal` 等字段 (DTO 定义里没有)，因此前端即使想读也没数据源。

2.1.2 修复方案 (分层)

(A) 后端： `RecordService` 的 `toView` / DTO 增加四性字段透出

在 `RecordDto` 与后端 `toView` 中补充 (读取节点 `properties` 的 `finance:checkReal/checkComplete/checkUsable/checkSafe/checkedAt`，并附带最近一次报告 `reportId`)：

```
// RecordDto 新增
checkReal: boolean;
checkComplete: boolean;
checkUsable: boolean;
checkSafe: boolean;
checkedAt?: string;
checkReportId?: string;
```

后端读取逻辑：在 `ctxOf / toView` 已有节点 `properties` 基础上，从 `finance:fourChecked` aspect 读出 4 个 `boolean` + 时间。

(B) 前端： `dtoToRecord` 用真实值替换硬编码

```
checks: {
  real: dto.checkReal ?? false,
  complete: dto.checkComplete ?? false,
  usable: dto.checkUsable ?? false,
  safe: dto.checkSafe ?? false,
},
checkDetails: [], // 明细如需展示，走 /inspection/reports?target=nodeId 异步补拉
checkedAt: dto.checkedAt,
checkReportId: dto.checkReportId,
```

`?? false` 兼容历史无字段数据 (未检测保持 `false`，语义正确)。

(C) 统计口径统一 (配套，一并修)

- `statsEngine.ts` 的 `computeLifecycle / computeCompliance` 中 `checksPassRate / checksByProperty / abnormalRecords` 现在用 `records[i].checks` 计算 → 打通后即可反映真实检测结果；
- 删除 `ComplianceStatsPage.tsx` 中 "四性检测异常档案" 的硬编码 `0`，改为读 `cc.abnormalRecords` (或后端 `/stats/lifecycle` 的 `inspectionPassRate`)；

- 明确口径：前端 `checksPassRate` 以 `r.checks` 为准，后端 `/stats/lifecycle` 的 `inspectionPassRate` 以报告表 `ams_inspection_report` 为准，两者以"已检测件"为分母统一，避免口径冲突。

2.1.3 涉及文件清单

文件	改动
<code>ams-server/.../record/RecordService.java</code>	<code>toView</code> 增加四性字段透出
<code>financeWeb/src/services/recordService.ts</code>	<code>RecordDto</code> + <code>dtoToRecord</code> 读取真实值
<code>financeWeb/src/utis/statsEngine.ts</code>	统计沿用真实 <code>checks</code> （无需大改，打通即生效）
<code>financeWeb/src/pages/archive-stats/ComplianceStatsPage.tsx</code>	删除硬编码 0，接真实数据

2.1.4 验收标准

1. 对一件已跑过四性检测的件，前端详情/统计页显示真实四性结论，不再恒为 `false/0%`；
2. 未检测件保持"未检测"（`false`）语义，不误判通过；
3. 前后端统计口径一致，`checksPassRate` 与 `/stats/lifecycle` 的 `inspectionPassRate` 在相同分母下数值一致。

2.2 断点③：修复配置页"立即执行检测"假通过

2.2.1 现状与问题根因

- `InspectionConfigPage.tsx` `handleRunBatch` 提交：
`ts '/inspection/run-batch', { fondsCode, phase: 'manual-config-page' }`
- 后端 `InspectionService.runBatch` → `run(nodeId, phase)` → `enabledItems('manual-config-page')` 查不到任何检测项（`ams_inspection_item.phase` 仅 `gd/yj/cq`）→ 返回空列表 → `finishReport` 中 `dimPass` 对空流 `allMatch=true` → 四维全通过 → `allPass=true`。

2.2.2 修复方案

根因：前端传了非法的 `phase` 值。最小且正确的修法是让配置页"立即执行检测"传一个合法的收集池环节，并让后端对非法 `phase` 做防御。

（A）前端修正（首选，一行改动）：

```
'/inspection/run-batch', { fondsCode, phase: 'gd' }
```

配置页"立即执行检测"针对的是收集池（未组卷）件，属归档（`gd`）环节语义最贴切。若希望跑"方案勾选的环节合集"，可改为传 `null`（后端 `runBatch` 已对 `null` 默认 `gd`），并明确告知用户当前批量检测口径为归档环节。

（B）后端防御加固（推荐一并做，防未来再传非法 `phase`）：

在 `InspectionService.run` 与 `runBatch` 中对 `phase` 做白名单校验：


```
if (phase != null && !List.of("gd","yj","cq").contains(phase)) {  
    // 记 warn 日志并回退到 gd，或直接抛 VALIDATION_FAILED  
    phase = "gd";  
}
```

这样即使前端误传，也不会产生"空结果假通过"的致命后果。核心原则：检测必须显式执行，宁可报错也不静默判通过。

2.2.3 涉及文件清单

文件	改动
financeWeb/src/pages/archive-config/InspectionConfigPage.tsx	phase: 'gd'
ams-server/.../inspection/InspectionService.java	run / runBatch 对 phase 做白名单防御
ams-server/.../inspection/InspectionController.java	(可选) runBatch 参数校验前移

2.2.4 验收标准

- 1. 配置页"立即执行检测"不再对"全烂档案"显示全部通过；
- 2. 对一件缺必填字段/格式不合规的件，批量检测正确判为"不通过"并列出问题；
- 3. 传非法 phase 时后端有明确防御（回退或报错），不静默通过。

第三部分 "原始凭证四性检测"接入方案（isRequired 落地到后端 + 检测项）

3.1 现状与问题根因

原始凭证（附件级，finance:sourceDocument）是目前司法采信权重最高、却完全未受检测的部分，存在三个断链：

断链	现状	后果
① 未持久化	SourceDocService.create 只写公共字段，不写 finance:extField	前端配置的扩展字段根本存不下来
② 必填约束失效	isRequired 仅存在于 sourceDocument.ts 的 extFieldDefs 与前端表单，后端 create 无任何必填校验	发票代码/税号等法律要件缺失检测不到
③ 不进检测引擎	InspectionService 只针对 finance:record，SourceDocService 无任何检测逻辑	原始凭证的 checks 字段形同虚设

3.2 总体目标

建立"原始凭证元数据驱动 → 后端强约束 + 四性检测项"的完整闭环，使原始凭证的必填法律要件、格式、哈希、四性状态均可被系统自动校验并留痕，支撑"官方认可、司法可采信"。

```
前端 extFieldDefs(isRequired)
  → 后端类型定义表(AMS)
  → create 时强制校验 + 持久化 extFields
  → 新增原始凭证四性检测项(sourcedoc 环节)
  → 检测结果写回 finance:sourceDocument 四性 aspect + 报告表
```

3.3 详细设计方案

3.3.1 阶段 A：扩展字段定义下沉到后端 (isRequired 落地)

A1. 后端建立"原始凭证类型-扩展字段定义"配置表

新增表 `ams_source_doc_type_field`（或复用 `ams_config` 以 JSON 存一份类型字典），把 `SOURCE_DOC_TYPE_TREE` 中的 `extFieldDefs` 迁移为后端可读取的结构化配置：

```
CREATE TABLE ams.ams_source_doc_type_field (
  doc_type_code text NOT NULL, -- 如 vat-special-invoice
  field_key text NOT NULL, -- 如 invoiceCode
  field_label text NOT NULL,
  data_type text NOT NULL, -- string/number/date/boolean/decimal
  is_required boolean NOT NULL,
  field_group text NOT NULL, -- basic/entity/business/amount/approval/attachment
  PRIMARY KEY (doc_type_code, field_key)
);
```

该表可由前端 `SOURCE_DOC_TYPE_TREE` 一次性 seed，或提供接口让元数据配置页写回。关键点：
`isRequired` 从"前端类型常量"变为"后端可查询配置"。

A2. `SourceDocService.create` 增加必填校验 + 持久化 extFields

在 `create` 中：

1. 读取 `docTypeCode` → 查 `ams_source_doc_type_field` 中 `is_required=true` 的字段；
2. 从入参 `fields` 取 `extFields`（JSON 字符串或 Map），逐项校验必填字段是否非空；
3. 任一必填缺失 → 抛 `BizException`（`REQUIRED_FIELD_MISSING` + 缺失字段清单），拒绝建件；
4. 校验通过后，将 `extFields` 序列化写入 `finance:extFields`（补上当前缺失的写入）。

```
// 伪代码
String docType = str(fields.get("docTypeCode"));
```

```

List<FieldDef> required = fieldRepo.requiredFieldsOf(docType);
Map<String,Object> ext = parseJson(fields.get("extFields"));
List<String> missing = required.stream()
    .filter(f -> isBlank(ext.get(f.key())))
    .map(FieldDef::label).toList();
if (!missing.isEmpty())
    throw BizException.badRequest("REQUIRED_FIELD_MISSING",
        "原始凭证必填字段缺失: " + String.join(", ", missing));
props.put("finance:extFields", toJson(ext));

```

A3. SourceDocController 上传接口透传 extFields

upload 的 fields 已是 @RequestParam Map<String,String>，需支持接收 extFields 的 JSON 串（Multipart 表单字段），交由 service 解析校验。

3.3.2 阶段 B：新增"原始凭证四性检测"检测项（并入检测引擎）

B1. 扩展检测环节维度

现有 ams_inspection_item.phase 仅 gd/yj/cq（CHECK 约束）。新增附件环节 sd（source-doc）：

```

ALTER TABLE ams.ams_inspection_item
    DROP CONSTRAINT ck_inspection_item_phase;
ALTER TABLE ams.ams_inspection_item
    ADD CONSTRAINT ck_inspection_item_phase CHECK (phase IN ('gd','yj','cq','sd'));

```

B2. 新增原始凭证检测项（seed 数据）

code	phase	dim	检测项	check_type	说明
SD-1-01	sd	real	附件文件存在性	sd-file-present	附件内容非空
SD-1-02	sd	real	附件哈希登记	sd-hash-registered	finance:digitalHash 非空（待扩展为真哈希比对）
SD-1-03	sd	real	电子签名校验	sd-signature-verified	SourceDocFile.signatureVerified
SD-2-01	sd	complete	扩展字段必填校验	sd-required-fields	读 ams_source_doc_type_field.is_required 逐项校验
SD-2-02	sd	complete	公共字段必填	sd-common-required	documentNo/transactionDate/amountLower 等
SD-2-03	sd	complete	附件与记账凭证绑定	sd-parent-link	parentVoucherNo/parentRecordId 非空
SD-3-01	sd	usable	附件格式白名单	sd-format-whitelist	OFD/PDF/XML/图片
SD-3-02	sd	usable	元数据可读	sd-metadata-readable	公共字段齐全
SD-4-01	sd	safe	敏感信息扫描	sd-sensitive-pattern	OCR 文本身份证/卡号

code	phase	dim	检测项	check_type	说明
SD-4-02	sd	safe	密级合规	sd-security-level-valid	继承父件密级

B3. InspectionService 扩展执行器

在 `execRecord` 增加 `case "sd-..."` 分支，读取 `finance:sourceDocument` 的公共字段 + `finance:extFields`，执行对应检查。其中 `sd-required-fields` 是核心新增，实现：

```
case "sd-required-fields": {
  String docType = str(p.get("finance:docTypeCode"));
  Map<String,Object> ext = parseJson(str(p.get("finance:extFields")));
  List<String> missing = fieldRepo.requiredFieldsOf(docType).stream()
    .filter(f -> isBlank(ext.get(f.key())))
    .map(FieldDef::label).toList();
  return new ItemResult(code, name, dim, missing.isEmpty(),
    missing.isEmpty() ? "" : "缺少必填扩展字段: " + String.join("、", missing), nodeId);
}
```

B4. 新增"按记账凭证/原始凭证"检测入口

在 `InspectionController` / `SourceDocController` 增加：

- POST `/inspection/run-sourcedoc`：对单个原始凭证件执行 `sd` 环节检测（或复用 `/inspection/run`，`phase='sd'`）；
- 批量：POST `/inspection/run-sourcedoc-batch` 遍历某记账凭证下全部原始凭证；
- 卷级联动：`runVolume` 在卷内件检测时，顺带对其子附件执行 `sd` 检测，附件任一必填缺失 → 该件完整性维度判不通过。

B5. 检测结果回写

对 `finance:sourceDocument` 节点，复用 `finance:fourChecked` aspect（`finance-model-current.xml` 已定义），写入 `checkReal/checkComplete/checkUsable/checkSafe/checkedAt` + 落一行 `target_kind='source-doc'` 的报告。

3.3.3 阶段 C：前端展示与统计

- `SourceDocumentSearchPage`：扩展详情面板展示每张原始凭证的四性结果（读 `checkReal` 等字段），并用 `def.isRequired` 高亮"必须项缺失"状态；
- `SourceDocMetadataPanel`：新增"后端强约束同步状态"提示，展示当前类型有多少 `isRequired` 项已下沉到后端；
- 统计页：原始凭证四性合格率单列，纳入合规统计，不再与记账凭证混算。

3.3.4 涉及文件清单

层	文件	改动
迁移	新增 <code>V9_source_doc_field.sql</code> / <code>V10_inspection_sd_items.sql</code>	建配置表 + 加 <code>sd</code> 环节 + <code>seed</code> 检测项

层	文件	改动
后端	SourceDocService.java	create 必填校验 + 写 extFields
后端	新增 SourceDocTypeFieldRepo	读 isRequired 配置
后端	SourceDocController.java	upload 透传 extFields
后端	InspectionService.java	sd 环节执行器 + run-sourcedoc 入口
后端	InspectionController.java	新端点
模型	finance-model-current.xml	已含 fourChecked aspect, 无需改; 如需可加 sourceDoc 专用 aspect
前端	recordService.ts / sourceDocumentService.ts	透出/读取四性字段
前端	SourceDocumentSearchPage.tsx / SourceDocMetadataPanel.tsx	四性展示 + isRequired 状态
前端	InspectionConfigPage.tsx	sd 环节检测项配置 UI

3.3.5 验收标准

- 1. 上传原始凭证时, 若发票类缺 invoiceCode / sellerTaxId 等必填 → 后端拒绝建件并返回缺失清单;
- 2. finance:extFields 正确持久化, 可被详情页/检索读取;
- 3. 对已建原始凭证跑 sd 检测, 缺必填的判 complete 不通过并列字段; 通过者四性写回 aspect + 报告表;
- 4. 记账凭证卷级检测能联动其子附件, 附件缺陷可上抛到卷级完整性结论。

附录 跨三部分的关键依赖关系

方案① 差异表(P0) → 给出原始凭证必填字段清单(isRequired 依据)

|

方案③ isRequired 落地 + sd 检测项 → 需要方案①字段清单作为配置表 seed

|

方案② 断点①(前端读检测结果) → 打通后 方案③ 的原始凭证四性结果才能被前端统计展示

|

└─ 三者共同构成"元数据合规 ↔ 四性检测"可信闭环, 支撑官方认可/司法采信

建议落地顺序: 先做方案② (修复正确性, 见效最快) → 再做方案①差异表 (固化字段清单) → 最后按方案③分阶段接入原始凭证四性检测。

(本方案仅涉及设计与改造说明，未包含任何代码实现，供评审与排期参考。)